// CYBER SECURITY PROJECT · BATCH-05 · FILE INTEGRITY MONITORING

FILE INTEGRITY CHECKER_

detect tampering with SHA-256 · modified / new / deleted

project.info

```
$ cat project.info
author    = "Faiq Umer"
id        = "ZYNVEX-CERT-2217"
project   = "File Integrity Checker"
language  = "Python 3 (standard library only)"
algorithm = "SHA-256"
tests     = "7 / 7 passing"
source    = "github.com/FaiqUmer/file-integrity-checker"
date      = "November 2026"
$ _
```

00 // INDEX

#	Section
01	Project overview and objectives
02	Security concepts
03	Features and technologies used
04	Installation and setup
05	Usage and screenshots
06	System workflow and architecture
07	Testing and results
08	Challenges and limitations
09	Future improvements
10	Conclusion
11	References

REPOSITORY
https://github.com/FaiqUmer/file-integrity-checker

01 // PROJECT OVERVIEW AND OBJECTIVES

The **File Integrity Checker** is a command-line tool that detects unauthorised changes to files. It first records a cryptographic fingerprint (a SHA-256 hash) of every file in a trusted folder. This record is called the **baseline**. On later runs the tool recomputes the fingerprints and compares them with the baseline, then reports every file that was **modified**, **newly added** or **deleted**.

This technique is called **File Integrity Monitoring (FIM)**. Security products use it to spot altered configuration files, planted web shells, trojaned binaries and wiped logs.

> Objectives

- Apply cryptographic hashing to verify file integrity.
- Detect modified, added and deleted files in a folder tree.
- Give clear output plus exit codes that scripts can use.
- Prove correctness with automated unit tests and a real-machine run.
- Document the design, limitations and future work.

02 // SECURITY CONCEPTS

> Integrity (the "I" in the CIA triad)

Integrity means data has not been changed in an unauthorised way. Monitoring file integrity is a recognised control in NIST SP 800-53 (SI-7) and PCI DSS (Requirement 11.5.2, change-detection mechanism).

> Cryptographic hash functions

SHA-256 (FIPS 180-4) maps any input to a 256-bit digest. The same input always gives the same digest. Changing even one bit gives a completely different digest. Finding a different file with the same digest is computationally infeasible. A changed digest therefore proves the content changed.

> Baseline comparison

Condition	Result	Possible attack indicator
In both scans, hash differs	MODIFIED [~]	Tampered config, trojaned binary
Only in current scan	NEW [+]	Web shell or malware dropped
Only in baseline	DELETED [-]	Log wiping, removed evidence

03 // FEATURES AND TECHNOLOGIES USED

SHA-256
Hashed in 8 KB chunks, so large files use little memory.

RECURSIVE
Scans every sub-folder; stores relative paths.

3 CHANGE TYPES
Modified, new and deleted files reported.

JSON BASELINE
Human-readable; custom path with `--baseline`.

SELF-AWARE
Baseline inside the folder is skipped automatically.

EXIT CODES
0 OK · 1 changes found · 2 error.

> Technology stack

Item	Purpose
`Python 3.8+`	Implementation language (verified on Python 3.13, Windows)
`hashlib`	SHA-256 hashing
`os`	Recursive folder traversal with `os.walk`
`json`	Baseline storage
`argparse`	Command-line interface
`unittest`	Automated tests
`Git / GitHub`	Version control and hosting

04 // INSTALLATION AND SETUP

No third-party packages are needed. Requirement: Python 3.8 or newer on Windows, Linux or macOS (`python --version` to check).

```
$ git clone https://github.com/FaiqUmer/file-integrity-checker.git
$ cd file-integrity-checker
$ python file_integrity_checker.py --help
```

Or download the ZIP from the GitHub page (**Code → Download ZIP**), unzip it and open a terminal in that folder.

05 // USAGE AND SCREENSHOTS

> Step 1: create the baseline

Run `init` on a folder you trust. One SHA-256 hash is stored per file.

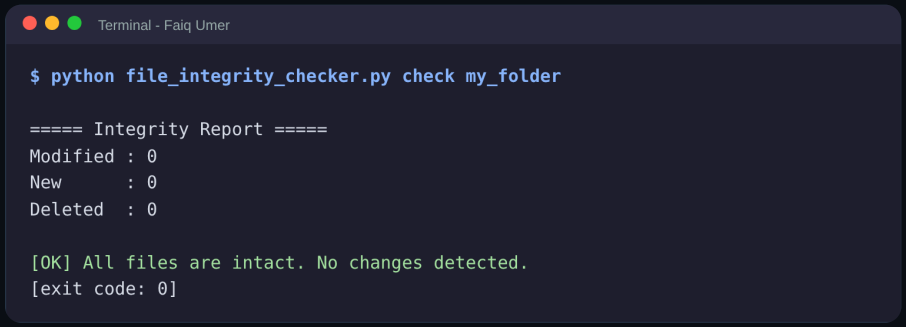


// Fig 1: baseline created for 4 files



// Fig 2: baseline.json maps each relative path to its SHA-256 hash

> Step 2: check an unchanged folder

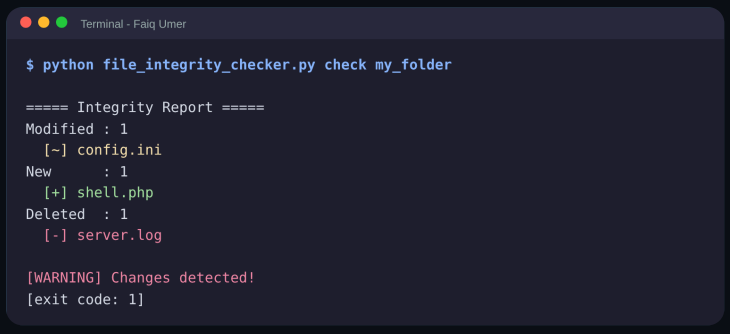


// Fig 3: no changes detected, exit code 0

> Step 3: simulate an attack and check again

One file was edited, one deleted and a suspicious `shell.php` added.

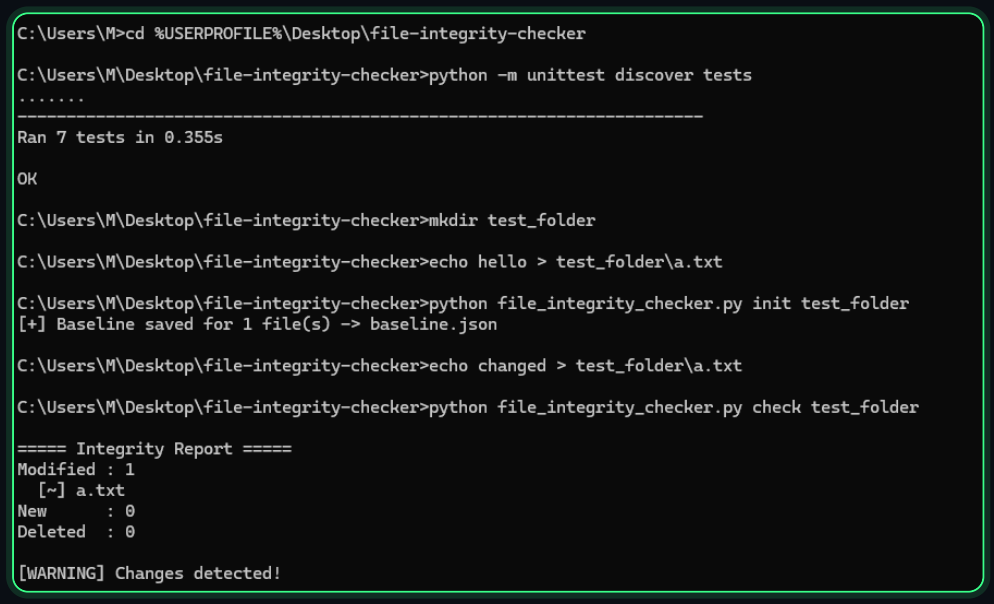
```
$ echo "admin:hacked" > my_folder/config.ini
$ rm my_folder/server.log
$ echo "backdoor" > my_folder/shell.php
```



// Fig 4: all three changes detected, exit code 1

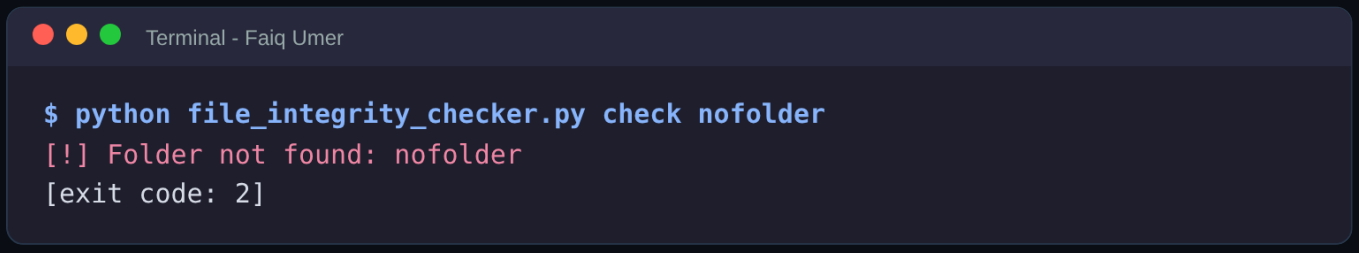
> Step 4: run on Windows (real machine)

The same workflow was run in Windows Command Prompt on the author's own computer: tests first, then `init`, a file edit, and `check`.



// Fig 5: Windows Command Prompt run, 7 tests OK, then [~] a.txt flagged as Modified

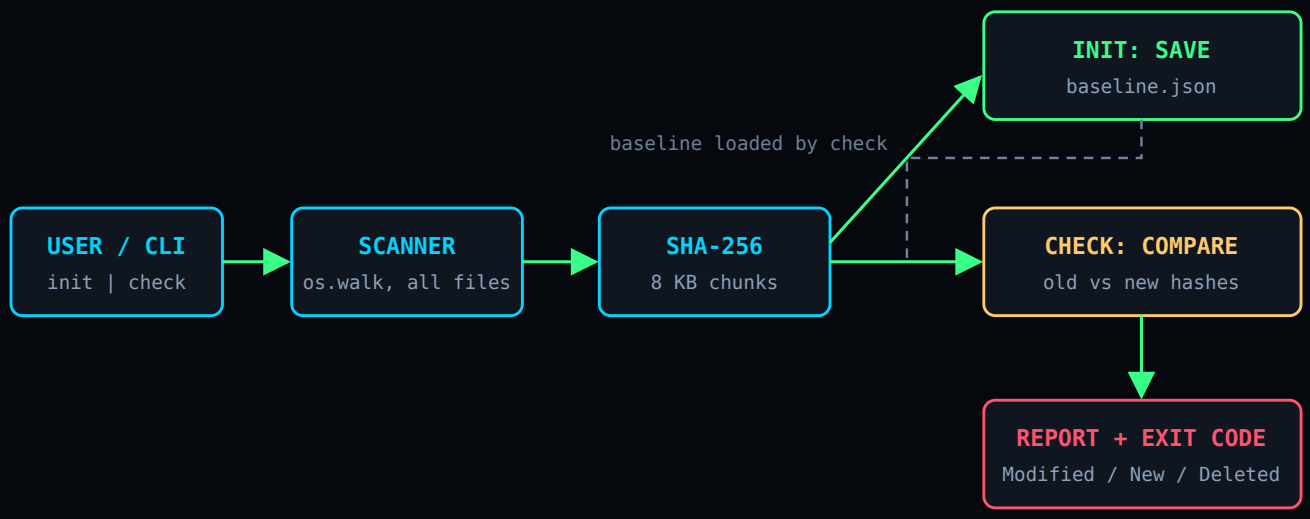
> Error handling



// Fig 6: missing folder gives a clear message and exit code 2

06 // SYSTEM WORKFLOW AND ARCHITECTURE

Four small parts make up the tool: a command-line interface, a folder scanner, a hashing function, and a compare-and-report stage. The baseline file links the two modes.



// Fig 7: workflow of the File Integrity Checker

> Flow

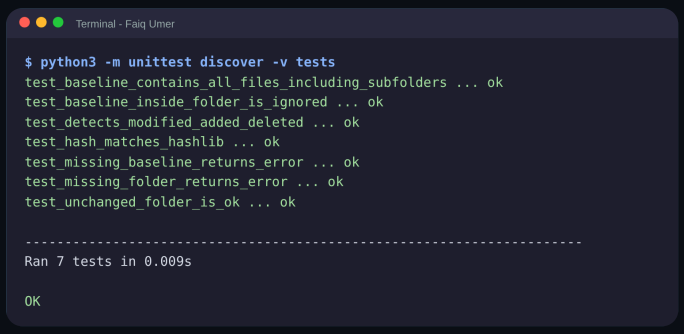
1. The user runs `init` or `check` with a folder path.
2. The scanner walks the folder recursively and opens each file in binary mode.
3. Each file is hashed with SHA-256 in 8 KB chunks.
4. **init:** `{relative_path: hash}` is written to `baseline.json`.
5. **check:** the baseline is loaded and compared with the new scan.
6. The report lists modified, new and deleted files. Exit code is 0 (no change) or 1 (changes).

> Key functions

Function	Role
`hash_file(path)`	SHA-256 hex digest of one file
`scan_folder(folder, skip)`	`{relative_path: hash}` for the whole folder
`create_baseline(folder, file)`	Saves the scan as JSON
`compare(old, new)`	Sorted modified, added and deleted lists
`check_integrity(folder, file)`	Loads baseline, compares, prints report
`main(argv)`	Parses arguments, returns the exit code

07 // TESTING AND RESULTS

Seven unit tests use Python's `unittest`. Each builds a temporary folder, so real data is never touched. All 7 pass, both in the development environment and on the author's Windows machine (Fig 5).



// Fig 8: unit test results, 7 tests OK

#	Test	Expected	Result
1	Hash equals `hashlib` reference	Digests match	PASS
2	Baseline includes sub-folder files	All 3 files listed	PASS
3	Unchanged folder checked	Exit code 0	PASS
4	One edited, one deleted, one added	1 modified, 1 new, 1 deleted; exit 1	PASS
5	Check without a baseline	Error, exit 2	PASS
6	Folder does not exist	Error, exit 2	PASS
7	Baseline stored inside monitored folder	Ignored; exit 0	PASS

RESULT

In the demonstration, the tool flagged the edited `config.ini` as modified, the injected `shell.php` as new and the removed `server.log` as deleted, with no false positives for the untouched files. On Windows, the edited `a.txt` was flagged as modified.

08 // CHALLENGES AND LIMITATIONS

> Challenges

- **Large files:** loading a whole file into memory is wasteful, so files are hashed in small chunks.
- **Baseline inside the folder:** it would show up as a "new" file, so it is skipped explicitly.
- **Moved folders:** relative paths keep the baseline valid if the folder moves.
- **Safe testing:** temporary directories keep tests isolated and repeatable.

> Limitations

- Anyone who can edit `baseline.json` can hide changes. Keep the baseline on read-only or separate storage.
- Only file content is checked, not permissions, owner or timestamps.
- Checks run on demand; this is not real-time monitoring.
- Files that change legitimately, such as logs, appear as modified.

09 // FUTURE IMPROVEMENTS

- Real-time monitoring (for example with the `watchdog` library).
- Protect the baseline with an HMAC or digital signature.
- Track permissions, ownership and modification time.
- Exclude patterns (such as `*.log`) and an "update baseline" command.
- Email or chat alerts, and scheduled runs (cron or Task Scheduler).
- A simple GUI or web dashboard.

10 // CONCLUSION

The File Integrity Checker shows how one cryptographic primitive, SHA-256, can become a practical security control. It reliably detects modified, new and deleted files, reports them clearly, and signals the result through exit codes so scripts can act on it. It needs only the Python standard library, is covered by automated tests, and was verified on a real Windows machine. It is a compact project, but it uses the same core idea as professional FIM products, and the limitations and improvements listed above show how it could grow into a stronger monitoring tool.

11 // REFERENCES

1. NIST. *FIPS PUB 180-4: Secure Hash Standard (SHS)*. 2015.
2. NIST. *SP 800-53 Rev. 5: Security and Privacy Controls for Information Systems and Organizations*, control SI-7. 2020.
3. PCI Security Standards Council. *PCI DSS v4.0*, Requirement 11.5.2.
4. Python Software Foundation. *hashlib - Secure hashes and message digests*. docs.python.org/3/library/hashlib.html
5. Python Software Foundation. *unittest - Unit testing framework*. docs.python.org/3/library/unittest.html
6. Python Software Foundation. *argparse - Parser for command-line options*. docs.python.org/3/library/argparse.html

SOURCE CODE
https://github.com/FaiqUmer/file-integrity-checker